DA3408 Assignment 5

Vrishab Anurag Venkataraghavan - DA24B033

1. Part A: REST service

A1, A2: `partA/eta_app.py` has the implementation, and `partA/test_eta_app.py` contains 34 REST test cases (including valid request, order, 18 invalid-field cases, missing field, malformed JSON, infeasible rule and boundaries, batch sizes 0/1/500/501, request-ID echo/generation on success and error, metrics, generic 500). **46 passed** from pytest (including the 12 gate tests).

A3: v1 on one uvicorn worker, 20 warm-up requests, then 4 s per concurrency level (`load_test.py`, closed loop). Throughput = requests / elapsed time.

Clients	req/s	mean ms	p50 ms	p95 ms	p99 ms	errors
1	499.6	2.00	1.5	3.8	4.8	0
2	454.9	4.38	4.1	6.8	8.3	0
4	540.2	7.39	6.6	12.2	14.2	0
8	557.6	14.30	13.7	22.4	27.9	0
16	521.3	30.51	30.1	45.3	56.7	0

Throughput stops growing at about 4 clients (455 to 558 req/s from 2 to 16 clients) while mean latency roughly doubles with every doubling of clients. A single Python process on one core is the limit. Little's Law $L = XW$ confirms this: 8 clients give $557.6 \times 0.01430 = 7.97$ and 16 clients give $521.3 \times 0.03051 = 15.9$ requests in flight, i.e. all clients are queueing.

Capacity under p95 $\leq$ 80 ms: Every level meets the SLO (largest p95 is 45.3 ms) and has no errors, so capacity is the highest throughput: **557.6 req/s** Saturation: **at 8 clients**. CPU was the limiting factor (not SLO).

Batching (concurrency 1):

Rows per request	mean ms per request	ms per row
1	1.995	1.995
100	3.029	0.0303

A batch costs about 66 times less per row. HTTP handling, JSON parsing, validation and DataFrame creation go once per request, and the model predicts all rows in one vectorised call.

Change: two uvicorn workers (same sweep, expected roughly 2x because the work is CPU-bound and one process uses one core).

Clients	req/s	mean ms	p95 ms	errors
1	470.6	2.12	4.3	0
2	991.0	2.01	4.1	0
4	1116.1	3.57	7.0	0
8	900.9	8.85	15.0	0
16	1139.1	13.96	25.8	0

Capacity: 557.6 -> **1139.1 req/s (2.04x)** at 16 clients with p95 25.8 ms. This is one 4 s sweep per level and is noisy. A second full run of the same experiment during verification gave 676 and 979 req/s (1.45x); not fully double capacity with the change.

Why p95/p99 is reported rather than mean? The mean does not truly show the extent of how slow requests can be, but these requests have real impact on users and SLOs. p95/p99 therefore are used to show queueing delay that is averaged and hidden by the mean.

2. Part B: MLflow

B1: `register()` in `partB/eta_mlflow.py` logs a custom pyfunc (columns `eta_minutes` rounded to one decimal and `model_version`) with a signature, an input example and pinned `pip_requirements`. SQLite tracking store, model `eta-model`. Included is the registry table

Version	Alias	Bundle
1	prod	v1 (Ridge)
2	candidate	v2 (GradientBoosting)

B2: `decide()` is pure; promotion needs all four criteria. `promotion_gate()` loads both aliases, scores `make_data(2000, seed=99)` and times 200 single-row predictions (after warm-up, no HTTP). `test_gate.py` has 12 cases as mentiooned earlier (inclusive boundaries just inside/outside each threshold, custom thresholds, invalid metrics, zero baselines, no input mutation).

Model	MAE	rush MAE	rain MAE	p95 ms
prod (v1)	11.955	19.680	11.623	0.758
candidate (v2)	6.105	8.580	6.285	1.580

Relative MAE gain is 48.94%.

Gate	gain	p95 $\leq$ 10 ms	rush slice	rain slice	promote
default (min gain 3%)	pass	pass	pass	pass	**true**
strict (min gain 60%)	fail	pass	pass	pass	**false**

B3: `mlflow models serve` for `prod` (port 5101) and `candidate` (5102); the same first 200 rows of `data/eta_fresh.csv`, one row per request, 10 warm-ups per server.

Mean absolute prediction difference	9.48 min
Requests differing by more than 10 min	35.0%
HTTP p95, prod / candidate	5.41 ms / 5.54 ms

Decision: v2 should likely be promoted, but only to a monitored 10% canary. It cuts MAE by 49% with no slice regression and no latency cost, and the large disagreement (35% of requests differ by more than 10 minutes) is expected from a much more accurate model. However this would be a big change for customers in a real world scenario, so the exposure or widespread deployment should be limited until real labelled errors can confirm the gain.

Why moving an alias does not change a running server: An alias is resolved to a concrete model version only when the server loads `models:/eta-model@prod`; the process keeps that model in memory and never looks at the registry again. The two ways to pick up a new version without dropping requests: (1) a rolling restart behind a load balancer, starting new servers, waiting for readiness and draining old ones; (2) blue/green, i.e. start the new version alongside, switch traffic, then retire the old one.

3. Part C: Ray Serve canary

`partC/eta_ray.py`: `EtaModel` loads its bundle once per replica; `prod` has 2 replicas and `candidate` has `autoscaling_config` min 1, max 3. A FastAPI-ingress `Router` takes `{"customer_id", "instances"}` and returns the model output plus `arm`. The arm is chosen from SHA-256 of the customer ID mapped to [0, 1); it goes to v2 if below `weight`. `X-Beta: 1` forces v2. Every replica and the router reserve 0.25 CPU, so the maximum of 5 model replicas plus the router needs 1.5 CPUs.

C2: `partC/verify_ray.py` (real HTTP, 2-CPU Ray, weight 0.2, IDs `customer-0000` to `customer-0999`):

Check	Result
(a) candidate share within 0.20 $\pm$ 0.04	PASS, 0.213

(b) 200 customers get the same arm on a second call	PASS, 200/200
(c) same after deleting and redeploying the identical app	PASS, 200/200
(d) weight raised to 0.5: no candidate customer moves back to prod	PASS, 0 moves
(e) header override for 50 prod customers	PASS, 50/50 to v2; next normal call back on v1

C3:

1. Python's `hash()` of a string is salted per process (`PYTHONHASHSEED`). Therefore different replicas, restarts or redeployments would give the same customer different buckets. However SHA-256 is deterministic everywhere so does not face this issue.
2. Since this is an ETA product, if a customer sees a different estimamte on each request would stop trusting the estimates offered by the product. Therefore for the experiment, a customer must stay in one arm so each customer's outcomes belong to one version and the arms are not contaminated by this error or bias; the customer should be the unit of our analysis, not the request.
3. `target_ongoing_requests=2` keeps each replica busy but not queueing (`max_ongoing_requests=4` is the hard cap per replica). `upscale_delay_s=2` reacts quickly to a load rise and `downscale_delay_s=15` avoids replica flapping. Maximum 3 candidate replicas keeps the total at 1.5 CPUs inside the 2-CPU budget. Prod stays at 2 replicas for availability.

4. Part D: release gate

`partD/smoke_eta.py --url ... --expect-version ... [--max-p95-ms 80]` exits 0 on success and 1 on failure. It has seven checks: version/finite positive output; longer distance gives a longer ETA (sanity); heavier traffic does not shorten the ETA (sanity); batch order; invalid input returns 422; request ID present; p95 $\leq$ 80 ms over 30 requests. Result against v1: all seven pass, p95 3.56 ms, exit 0. With `--expect-version wrong` it prints `pass: false` and exits 1.

Abort criteria for the 10% canary of v2:

Metric	Abort threshold	Window
5xx or timeout rate	> 1% (or > 2x prod)	5 min, at least 200 canary requests
Request p95 latency	> 80 ms	two consecutive 5 min windows
Labelled MAE (overall, rush, rain)	> 5% worse than prod on the same customers	rolling 24 h, at least 100 labels per slice
Invalid outputs (NaN/inf, non-positive ETA, wrong count)	any occurrence	immediate

5. What's not been tested, limits and attribution

Not tested: production traffic and real labelled accuracy; sustained-load autoscaling of the candidate (only a functional check of routing); live alerting and rollback (the abort criteria are proposals); smoke test against v2 (only v1); other operating systems or Python versions; multi-machine deployment. Data is synthetic, load runs are short (4 s) and share the machine with the client.

Attribution: the assignment brief and `Module4_Assignment_Starter`. `eta_data.py` is unchanged; `load_test.py` was extended to report mean latency and elapsed-time throughput and to count only successful responses per replica. MLflow pyfunc and Ray Serve documentation were used for the APIs. More informative AI disclosure included in the README.